

1- True or False:-

a. True

b. True

c. False

d. False

e. True

f. False

2.

```
graph = {
    'S': ['B', 'D', 'A'],
    'A': ['C'],
    'B': ['G', 'D'],
    'C': ['G', 'D'],
    'D': ['G'],
    'G': []
}

def bfs(graph, start, goal):
    visited = []
    queue = [[start]]

    while queue:
        path = queue.pop(0)
        node = path[-1]

        if node in visited:
            continue

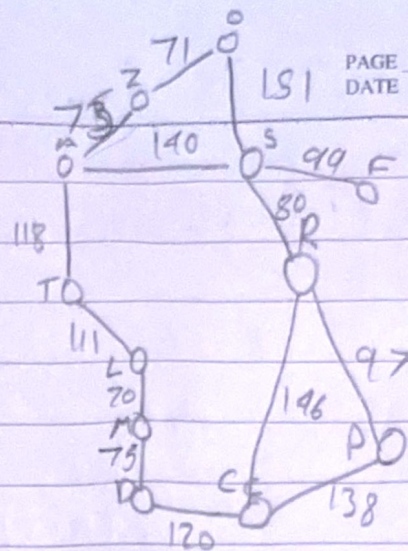
        visited.append(node)

        if node == goal:
            return path

        else:
            adjacent_nodes = graph.get(node, [])
            for node2 in adjacent_nodes:
                new_path = path.copy()
                new_path.append(node2)
                queue.append(new_path)
```

3. Graph Search- BFS

Q-	States	Queue
		[A]
	[A]	[A,S], [A,T], [A,Z]
	[A,S]	[A,T], [A,Z], [A,S,R], [A,S,F]
	[A,T]	[A,Z], [A,S,R], [A,S,F], [A,T,L]
	[A,Z]	[A,S,R], [A,S,F], [A,T,L]
		[A,Z,O]
	[A,S]	[A,S,F], [A,T,L], [A,Z,O]
		[A,S,R,P], [A,S,R,C]



[A,S,R,C]

Path cost = 140 + 80 + 146 = 366

DFS

Goal Reached!

b-	States	stack (FIFO) →
		[A]
	[A]	[A,Z], [A,T], [A,S]
	[A,S]	[A,Z], [A,T], [A,S,R]
	[A,S,R]	[A,Z], [A,T], [A,S,R,P], [A,S,R,C]
	[A,S,R]	Goal Reached!

[A,S,R,C]

Path cost = 140 + 80 + 146 = 366

UCS

c-	States	Priority Queue
		[A]
	[A]	[A,Z], [A,T], [A,S]
	[A,Z]	[A,T], [A,S], [A,Z,O]
	[A,T]	[A,S], [A,Z,O], [A,T,L]
	[A,S]	[A,Z,O], [A,S,R], [A,T,L], [A,S,F]
	[A,S,R]	[A,T,L], [A,S,F], [A,S,R,P], [A,S,R,C]

[A,S,R,C]

Path cost = 140 + 80 + 146 = 366.

[A,Z,O] → Dead-end or loops (F) A,Z,O,S,A

→ goal reached. If we checked all other possible paths, we find that this path [A,S,R] has the lowest cost of 366.

$\delta = A^*$ Search - $\text{node cost} \rightarrow \text{heuristic cost}$
Minimize $F(n) = g(n) + h(n)$

States	Parents	$F(n)$
[A]	[A]	(340)
[A,S], [A,T], [A,Z]	[A]	(355), (428), (425)
[A,S], [A,T], [A,Z], [A,S,R], [A,S,F]	[A,S], [A,T], [A,Z]	(366), (428), (425), (366), (449)
[A,S,R], [A,T], [A,Z], [A,S,F], [A,S,R,C], [A,S,R,P]	[A,S,R], [A,T], [A,Z], [A,S,F]	(366), (428), (425), (449), (366), (455)
[A,S,R] Goal Reached!		

Path cost = 366
[A,S,R,C]

4. The main advantage of the A^* is that it uses a heuristic function that estimates the distance to the goal, allowing the A^* to make a targeted search toward the goal instead of searching based only on the path cost as in UCS.

-Also A^* is generally faster because it covers the search space effectively.

5-



5-

```
import heapq

def a_star_search(graph, heuristics, start, goal):
    open_list = []
    heapq.heappush(open_list, (0 + heuristics[start], start, [start], 0))

    visited = set()

    while open_list:
        f, current_node, path, g_score = heapq.heappop(open_list)

        if current_node == goal:
            return path, g_score

        if current_node in visited:
            continue
        visited.add(current_node)

        if current_node in graph:
            for neighbor, distance in graph[current_node].items():
                if neighbor not in visited:
                    new_g = g_score + distance
                    new_f = new_g + heuristics.get(neighbor, float('inf'))
                    new_path = path + [neighbor]
                    heapq.heappush(open_list, (new_f, neighbor, new_path, new_g))

    return None, float('inf')

graph_map = {
    'A': {'Z': 75, 'S': 140, 'T': 118},
    'S': {'A': 140, 'F': 99, 'R': 80},
    'R': {'S': 80, 'P': 97, 'C': 146},
    'C': {'R': 146, 'D': 120, 'P': 138},
    'T': {'A': 118, 'L': 111},
}

heuristics_table = {
    'A': 340, 'B': 190, 'C': 0, 'D': 120, 'E': 370,
    'F': 210, 'G': 150, 'H': 390, 'I': 500, 'L': 200,
    'M': 170, 'N': 540, 'O': 380, 'P': 138, 'R': 146,
    'S': 215, 'T': 310, 'U': 280, 'V': 450, 'Z': 350
}

path, cost = a_star_search(graph_map, heuristics_table, 'A', 'C')
print(f"Path: {path}")
print(f"Total Cost: {cost}")
✓ [1] < 10 ms
```

Output:

```
44 path, cost = a_star_search(graph_map, heuristics_table, 'A', 'C')
45 print(f"Path: {path}")
46 print(f"Total Cost: {cost}")
```

✓ [1] < 10 ms

Path: ['A', 'S', 'R', 'C']

Total Cost: 366